

Fault-Tolerant Data Processing System

Hiring Task — Full Design & Implementation Guide

Duration: 60 minutes | **Note:** Completion is not expected. Judgment, structure, and trade-offs matter more than completeness. You may use AI assistants.

Problem Statement

Build a data ingestion and processing service that receives events from multiple external clients. Clients do not follow a strict schema, change formats without notice, may resend events, and may fail mid-request.

Your system must:

- Normalize incoming data
- Avoid duplicate processing
- Handle partial failures safely
- Provide consistent aggregated outputs

1. Event Ingestion — Raw Format

Clients send events in the following unreliable format:

```
{
  "source": "client_A",
  "payload": {
    "metric": "value",
    "amount": "1200",
    "timestamp": "2024/01/01"
  }
}
```

Important notes:

- Field names may differ across clients
- Types may be inconsistent (strings instead of numbers, dates in different formats)
- Some fields may be missing
- Clients may add extra fields without notice

2. Normalisation Layer

Convert raw events into a canonical internal format. Handle missing/malformed fields gracefully, avoid breaking on new fields, and clearly separate raw input from normalized data.

Canonical Format:

```
{
  "client_id": "client_A",
  "metric": "value",
  "amount": 1200,
  "timestamp": "2024-01-01T00:00:00Z"
}
```

normalizer.py — Core Normalization Logic

```
import hashlib, json
```

[illegible]

```

key_material = json.dumps(
    {k: normalized[k] for k in ("client_id", "metric", "amount", "timestamp")},
    sort_keys=True
)
normalized["idempotency_key"] = hashlib.sha256(
    key_material.encode()
).hexdigest()

return normalized

```

validator.py — Validation Layer

```

from dataclasses import dataclass, field
from typing import Optional

@dataclass
class ValidationResult:
    valid: bool
    errors: list[str] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)

REQUIRED_FIELDS = ["client_id", "metric", "timestamp"]

def validate(normalized: dict) -> ValidationResult:
    errors, warnings = [], []

    for f in REQUIRED_FIELDS:
        if normalized.get(f) is None:
            errors.append(f"Missing required field: {f}")

    if normalized.get("amount") is None:
        warnings.append("amount is None – will be stored as NULL")

    if normalized.get("timestamp") is None:
        errors.append("timestamp could not be parsed")

    return ValidationResult(valid=len(errors) == 0, errors=errors, warnings=warnings)

```

3. Idempotency & Deduplication

Clients may resend events with no guaranteed unique ID and unreliable timestamps. The system generates a deterministic SHA-256 idempotency_key from the normalized fields, then uses a database UNIQUE constraint to silently discard exact duplicates.

idempotency.py

```

"""
Idempotency strategy
=====
1. normalizer.py produces a SHA-256 idempotency_key from
   (client_id, metric, amount, timestamp).
2. The DB table has UNIQUE(idempotency_key).
3. On insert we use INSERT OR IGNORE (SQLite) / INSERT ... ON CONFLICT DO NOTHING (Postgres).
4. The API response is identical on first insert and on duplicates,
   so retrying clients observe the same behaviour.

Trade-offs
-----
- Two events with identical fields but different semantics collapse into one.

```

Mitigation: ask clients to include a nonce/sequence if they need true uniqueness.
 - If a timestamp cannot be parsed it defaults to None, widening the collision window.
 Mitigation: treat missing-timestamp events as always-insert (lower dedup guarantee).
 """

```
INSERT_SQL_SQLITE = """
    INSERT OR IGNORE INTO events
        (idempotency_key, client_id, metric, amount, timestamp, raw)
    VALUES (?, ?, ?, ?, ?, ?)
    """
```

```
INSERT_SQL_POSTGRES = """
    INSERT INTO events
        (idempotency_key, client_id, metric, amount, timestamp, raw)
    VALUES (%s, %s, %s, %s, %s, %s)
    ON CONFLICT (idempotency_key) DO NOTHING
    """
```

Database Schema

```
-- SQLite (swap types for Postgres / MySQL as needed)
CREATE TABLE events (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    idempotency_key TEXT    NOT NULL UNIQUE,    -- SHA-256 of canonical fields
    client_id     TEXT     NOT NULL,
    metric        TEXT,
    amount        REAL,
    timestamp     TEXT,                        -- ISO-8601, UTC
    raw           TEXT,                        -- original JSON blob
    received_at   TEXT    DEFAULT (datetime('now')),
    status        TEXT    DEFAULT 'processed' -- processed | rejected
);

CREATE INDEX idx_events_client ON events (client_id);
CREATE INDEX idx_events_ts    ON events (timestamp);
CREATE INDEX idx_events_status ON events (status);

-- Rejected / failed events land here instead
CREATE TABLE rejected_events (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    raw           TEXT,
    reason        TEXT,
    received_at   TEXT    DEFAULT (datetime('now'))
);
```

4. Partial Failure Handling

Scenario: event received → validated → DB write fails → client retries. The idempotency key + ON CONFLICT DO NOTHING ensures the retry is a no-op once the write eventually succeeds. If the write never succeeds the event is not counted.

app.py — Flask Ingestion Endpoint

```
from flask import Flask, request, jsonify
import sqlite3, json, os
from normalizer import normalize
from validator import validate

app = Flask(__name__)
```

```

DB = os.getenv("DB_PATH", "events.db")

def get_db():
    conn = sqlite3.connect(DB)
    conn.row_factory = sqlite3.Row
    return conn

@app.post("/ingest")
def ingest():
    simulate_failure = request.args.get("simulate_failure", "false").lower() == "true"

    try:
        raw = request.get_json(force=True)
    except Exception:
        return jsonify({"status": "error", "message": "Invalid JSON"}), 400

    normalized = normalize(raw)
    result = validate(normalized)

    if not result.valid:
        _store_rejected(raw, result.errors)
        return jsonify({
            "status": "rejected",
            "errors": result.errors,
        }), 422

    if simulate_failure:
        return jsonify({"status": "error", "message": "Simulated DB failure"}), 500

    try:
        conn = get_db()
        conn.execute("""
            INSERT OR IGNORE INTO events
                (idempotency_key, client_id, metric, amount, timestamp, raw)
            VALUES (?, ?, ?, ?, ?, ?)
        """, (
            normalized["idempotency_key"],
            normalized["client_id"],
            normalized["metric"],
            normalized["amount"],
            normalized["timestamp"],
            json.dumps(raw),
        ))
        conn.commit()
        conn.close()
    except Exception as e:
        return jsonify({"status": "error", "message": str(e)}), 500

    return jsonify({
        "status": "ok",
        "idempotency_key": normalized["idempotency_key"],
        "warnings": result.warnings,
    }), 200

def _store_rejected(raw: dict, errors: list):
    try:
        conn = get_db()
        conn.execute(
            "INSERT INTO rejected_events (raw, reason) VALUES (?, ?)",

```

```

        (json.dumps(raw), "; ".join(errors))
    )
    conn.commit()
    conn.close()
except Exception:
    pass # best-effort; don't let rejection storage crash the response

```

5. Query & Aggregation API

```

@app.get("/aggregates")
def aggregates():
    """
    Query params:
        client    - filter by client_id
        from_ts   - ISO-8601 start timestamp (inclusive)
        to_ts     - ISO-8601 end timestamp (inclusive)
    """
    client = request.args.get("client")
    from_ts = request.args.get("from_ts")
    to_ts = request.args.get("to_ts")

    sql = "SELECT client_id, metric, COUNT(*) as count, SUM(amount) as total FROM events WHERE 1=1"
    params = []

    if client:
        sql += " AND client_id = ?"; params.append(client)
    if from_ts:
        sql += " AND timestamp >= ?"; params.append(from_ts)
    if to_ts:
        sql += " AND timestamp <= ?"; params.append(to_ts)

    sql += " GROUP BY client_id, metric ORDER BY client_id, metric"

    conn = get_db()
    rows = conn.execute(sql, params).fetchall()
    conn.close()

    return jsonify([dict(r) for r in rows])


@app.get("/events")
def list_events():
    """Return recently processed and rejected events for the UI."""
    conn = get_db()
    processed = conn.execute(
        "SELECT id, client_id, metric, amount, timestamp, received_at "
        "FROM events ORDER BY id DESC LIMIT 100"
    ).fetchall()
    rejected = conn.execute(
        "SELECT id, raw, reason, received_at "
        "FROM rejected_events ORDER BY id DESC LIMIT 50"
    ).fetchall()
    conn.close()
    return jsonify({
        "processed": [dict(r) for r in processed],
        "rejected": [dict(r) for r in rejected],
    })

```

```

import sqlite3, os

DB = os.getenv("DB_PATH", "events.db")

def init():
    conn = sqlite3.connect(DB)
    conn.executescript("""
        CREATE TABLE IF NOT EXISTS events (
            id                INTEGER PRIMARY KEY AUTOINCREMENT,
            idempotency_key    TEXT NOT NULL UNIQUE,
            client_id          TEXT NOT NULL,
            metric             TEXT,
            amount             REAL,
            timestamp          TEXT,
            raw                TEXT,
            received_at        TEXT DEFAULT (datetime('now')),
            status             TEXT DEFAULT 'processed'
        );
        CREATE INDEX IF NOT EXISTS idx_events_client ON events (client_id);
        CREATE INDEX IF NOT EXISTS idx_events_ts      ON events (timestamp);

        CREATE TABLE IF NOT EXISTS rejected_events (
            id                INTEGER PRIMARY KEY AUTOINCREMENT,
            raw               TEXT,
            reason            TEXT,
            received_at       TEXT DEFAULT (datetime('now'))
        );
    """)
    conn.commit()
    conn.close()
    print(f"Database initialised: {DB}")

if __name__ == "__main__":
    init()

```

6. Frontend — Minimal UI

A single HTML file with vanilla JS. Submit raw events, toggle simulate-failure, and view processed / rejected events and aggregates.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Event Ingestion Dashboard</title>
  <style>
    * { box-sizing: border-box; font-family: system-ui, sans-serif; }
    body { margin: 0; background: #f0f2f5; color: #1a1a2e; }
    header { background: #1a1a2e; color: #fff; padding: 16px 24px; }
    header h1 { margin: 0; font-size: 1.3rem; }
    .container { max-width: 1000px; margin: 24px auto; padding: 0 16px; }
    .card { background: #fff; border-radius: 10px; padding: 20px;
      margin-bottom: 20px; box-shadow: 0 1px 4px rgba(0,0,0,.08); }
    h2 { margin: 0 0 14px; font-size: 1rem; color: #0f3460; }
    textarea { width: 100%; height: 130px; font-family: monospace;
      font-size: .85rem; border: 1px solid #ccd; border-radius: 6px;
      padding: 10px; resize: vertical; }
    .row { display: flex; gap: 10px; align-items: center; margin-top: 10px; }
    button { padding: 9px 20px; background: #0f3460; color: #fff;

```

```

        border: none; border-radius: 6px; cursor: pointer; font-size: .9rem; }
button:hover { background: #16213e; }
label { display: flex; align-items: center; gap: 6px; font-size: .9rem; cursor: pointer; }
#status { margin-top: 10px; padding: 8px 12px; border-radius: 6px;
        font-size: .85rem; display: none; }
.ok { background: #e8f5e9; color: #2e7d32; }
.err { background: #fce4ec; color: #b71c1c; }
table { width: 100%; border-collapse: collapse; font-size: .83rem; }
th { background: #f4f6f8; padding: 8px; text-align: left;
      border-bottom: 2px solid #e0e0e0; }
td { padding: 7px 8px; border-bottom: 1px solid #eee; vertical-align: top; }
tr:hover td { background: #fafbff; }
.badge { display: inline-block; padding: 2px 8px; border-radius: 12px;
        font-size: .75rem; font-weight: 600; }
.badge-ok { background: #c8e6c9; color: #1b5e20; }
.badge-rej { background: #ffcdd2; color: #b71c1c; }
</style>
</head>
<body>
<header><h1>&#x26A1; Fault-Tolerant Event Dashboard</h1></header>
<div class="container">

  <!-- Submission card -->
  <div class="card">
    <h2>Submit Raw Event</h2>
    <textarea id="payload">{
"source": "client_A",
"payload": {
  "metric": "revenue",
  "amount": "1200",
  "timestamp": "2024/01/15"
}
    }</textarea>
    <div class="row">
      <button onclick="submitEvent()">Send Event</button>
      <label>
        <input type="checkbox" id="simFail"> Simulate DB Failure
      </label>
    </div>
    <div id="status"></div>
  </div>

  <!-- Processed events -->
  <div class="card">
    <h2>&#x2705; Processed Events <button style="float:right;padding:5px 12px"
      onclick="loadEvents()">Refresh</button></h2>
    <table id="proc-table">
      <thead><tr>
        <th>ID</th><th>Client</th><th>Metric</th>
        <th>Amount</th><th>Timestamp</th><th>Received</th>
      </tr></thead>
      <tbody id="proc-body"><tr><td colspan="6">Loading...</td></tr></tbody>
    </table>
  </div>

  <!-- Rejected events -->
  <div class="card">
    <h2>&#x274C; Rejected / Failed Events</h2>
    <table>

```



```

        <thead><tr><th>ID</th><th>Reason</th><th>Raw</th><th>Received</th></tr></thead>
        <tbody id="rej-body"><tr><td colspan="4">Loading...</td></tr></tbody>
    </table>
</div>

<!-- Aggregates -->
<div class="card">
    <h2>&#x1F4CA; Aggregates <button style="float:right;padding:5px 12px"
        onclick="loadAggregates()">Refresh</button></h2>
    <table>
        <thead><tr><th>Client</th><th>Metric</th><th>Count</th><th>Total</th></tr></thead>
        <tbody id="agg-body"><tr><td colspan="4">Loading...</td></tr></tbody>
    </table>
</div>
</div>

<script>
const API = ""; // same origin; adjust if backend is on a different port

async function submitEvent() {
    const raw = document.getElementById("payload").value;
    const sim = document.getElementById("simFail").checked;
    const el = document.getElementById("status");
    el.style.display = "none";
    try {
        const res = await fetch(`${API}/ingest?simulate_failure=${sim}`, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: raw,
        });
        const data = await res.json();
        el.className = res.ok ? "ok" : "err";
        el.textContent = JSON.stringify(data, null, 2);
        el.style.display = "block";
        loadEvents(); loadAggregates();
    } catch(e) {
        el.className = "err";
        el.textContent = "Request failed: " + e.message;
        el.style.display = "block";
    }
}

async function loadEvents() {
    const res = await fetch(`${API}/events`);
    const data = await res.json();

    const pb = document.getElementById("proc-body");
    pb.innerHTML = data.processed.length === 0
        ? '<tr><td colspan="6">No events yet.</td></tr>'
        : data.processed.map(e => `<tr>
            <td>${e.id}</td><td>${e.client_id}</td><td>${e.metric||'-'}</td>
            <td>${e.amount||'-'}</td><td>${e.timestamp||'-'}</td>
            <td>${e.received_at}</td></tr>`).join("");

    const rb = document.getElementById("rej-body");
    rb.innerHTML = data.rejected.length === 0
        ? '<tr><td colspan="4">No rejections.</td></tr>'
        : data.rejected.map(r => `<tr>
            <td>${r.id}</td><td>${r.reason}</td>

```

```

        <td style="font-family:monospace;font-size:.75rem">${r.raw.slice(0,80)}...</td>
        <td>${r.received_at}</td></tr>`).join("");
    }

    async function loadAggregates() {
        const res = await fetch(`${API}/aggregates`);
        const data = await res.json();
        const ab = document.getElementById("agg-body");
        ab.innerHTML = data.length === 0
            ? '<tr><td colspan="4">No data.</td></tr>'
            : data.map(r => `<tr>
                <td>${r.client_id}</td><td>${r.metric||'-'}</td>
                <td>${r.count}</td><td>${r.total??'-'}</td></tr>`).join("");
    }

    loadEvents(); loadAggregates();
</script>
</body>
</html>

```

README — Required Questions

What assumptions did you make?

Events always arrive as JSON. SQLite is acceptable for a single-process prototype (swap for Postgres in production). The idempotency key is derived from (client_id, metric, amount, timestamp); two events that share all four values are considered identical. Timestamps that cannot be parsed are treated as required fields and the event is rejected. No authentication layer is implemented per the spec.

How does your system prevent double counting?

A SHA-256 hash of the four canonical fields becomes the idempotency_key, stored with a UNIQUE constraint. Every INSERT uses ON CONFLICT DO NOTHING / INSERT OR IGNORE, so retried identical events are silently discarded at the database level without affecting aggregates. The HTTP response is identical for first-insert and duplicate, so clients observe no difference.

What happens if the database fails mid-request?

The Flask handler wraps the INSERT in a try/except and returns HTTP 500. The event is not counted. On retry the client re-submits; the idempotency key guarantees the write is a safe no-op if the first attempt partially succeeded. Rejected events are stored in a separate rejected_events table on a best-effort basis.

What would break first at scale?

SQLite's single-writer lock becomes the bottleneck immediately under concurrent load. Replacing it with Postgres and a connection pool solves that. Next, a single Flask process saturates CPU on normalisation; a queue (Redis + Celery / Kafka) would decouple ingestion from processing. The SHA-256 dedup key also widens collision risk if clients send structurally identical but semantically distinct events — a client-supplied nonce field would tighten this.

Project File Structure

/

```
■■■ app.py           # Flask API – ingestion + query endpoints
■■■ normalizer.py    # Raw → canonical conversion + idempotency key
■■■ validator.py     # Required-field + type validation
■■■ idempotency.py   # Dedup strategy notes + SQL constants
■■■ init_db.py       # One-time DB schema setup
■■■ index.html       # Frontend (serve via Flask static or open directly)
■■■ events.db        # SQLite file (auto-created on first run)
■■■ README.md        # This document
```

Quick Start

```
pip install flask
python init_db.py
python app.py
# Open index.html in a browser (same origin as Flask on :5000)
```

Evaluation Criteria

We are evaluating: System thinking, Data modeling, Failure handling, and Ability to explain decisions. We are NOT evaluating speed, UI polish, or framework knowledge.

Optional extension: If time permits, consider schema evolution — old data must remain queryable as formats change. Strategy: keep the raw JSON blob intact in the events table and re-normalise on read using a versioned normalizer registry.